Template for Technical Solution Doc - AFF PRCNG TECHSOL

<Feature / Initiative Name> - AFF PRCNG TECHSOL

> **How to use this page**
>
> Use this template to explain a Frontend technical solution before implementation. Keep it straightforward. Prefer tables, diagrams, and bullet points over long paragraphs.
>
> The goal is not to document every file or every code detail. The goal is to make the solution clear enough for FE, BE, QA, PM, and future maintainers.
>
> **Suggested usage**
>
> - **Small FE task:** fill required sections only.
> - **Medium FE task:** fill most sections, but keep Performance, Security, and Rollout concise.
> - **Large / cross-team / shared module task:** fill all sections, especially API mapping, props contract, testing, rollout, and trade-offs.

---

0. Document Info

Field	Value
Status	Draft / In Review / Approved / Implemented
Owner	
Reviewers	FE TL, BE, QA, PM, Design
Related Jira	
Related Figma	
Related PRD / Requirement	
Related API Contract	

Related Storybook	
Target Release	
Last Updated	

> **Guideline**
>
> Keep this section short. The purpose is to help readers know the document status, owner, reviewers, and related references. Do not explain the solution here.

---

1. Executive Summary

- **Problem:**
- **Selected approach:**
- **Main FE changes:**
- **Main dependencies:**
- **Main risk:**
- **Rollout plan:**

> **Guideline**
>
> This section should be understandable in 1 minute. Write this for stakeholders who do not have time to read the full document.
>
> Avoid implementation details here. Focus on what problem we solve, what approach we choose, and what impact it has.
>
> **Good example**
>
> “We will introduce a shared promo component that can be consumed by SRP and PRP pages. The component will receive normalized promo data from the page API and handle loading, empty, error, and tracking states consistently.”
>
> **Bad example**
>
> “We will create `PromoCard.tsx`, add `usePromoQuery`, and pass `isLoading` from React Query.”

---

2. Background & Problem

Current Condition

-

Problem

-

Why Now

-

Affected Pages / Journeys

-
-

> **Guideline**
>
> Explain the current situation before proposing the solution. Focus on the business or engineering pain.
>
> Good topics to include:
>
> - Current limitation
> - Duplicate logic
> - Inconsistent UI behavior
> - API contract mismatch
> - Performance issue
> - Operational or maintenance issue
> - Cross-vertical inconsistency
>
> Avoid jumping directly to implementation. Avoid generic wording like “to improve UX” without explaining the actual pain.

Optional Current Flow Diagram

Use this only if the current flow is confusing or involves multiple systems.

```
flowchart LR
  User[User] --> Page[Current Page]
  Page --> API1[API A]
  Page --> API2[API B]
  Page --> UILogic[Page-specific UI Logic]
  UILogic --> UI[Rendered UI]

```

---

3. Goals & Non-Goals

Goals	Non-Goals

> **Guideline**
>
> Use this section to control scope.
>
> **Goals** explain what this solution must achieve.
>
> **Non-goals** explain what this solution intentionally will not solve.
>
> This is useful when stakeholders may assume additional scope.
>
> **Good non-goal example**
>
> “This project will not redesign the entire promo placement UI.”

---

4. Scope & Impacted Areas

Area	Impact	Owner	Notes

> **Guideline**
>
> Mention impacted pages, modules, packages, services, APIs, tracking, or tests.
>
> Do not list every changed file unless the file is critical.
>
> **Good level of detail**
>
> - SRP Page
> - PRP Page
> - Shared Promo Component
> - Pricing API Contract
> - Tracking Event
> - E2E Test
>
> **Too detailed**
>
> - `src/components/Card/index.tsx`
> - `src/utils/formatter.ts`
> - `src/hooks/useSomething.ts`

Optional Context Diagram

Add this if the feature touches multiple modules, APIs, packages, or verticals. Skip it for small UI-only changes.

```
flowchart LR
  Page[Next.js Page] --> Feature[Feature Module]
  Feature --> Adapter[API Adapter]
  Adapter --> API[Backend API]
  Feature --> Tracking[Tracking]
  Feature --> SharedPackage[Shared Package / Component]

```

---

5. Proposed Solution

5.1 Solution Overview

We will implement `<feature / solution>` by:

1.
2.
3.

The main idea is:

-

> **Guideline**
>
> This is the most important section. Explain the selected approach clearly before going into detailed component, API, or state design.
>
> Keep it high-level but concrete.
>
> **Good example**
>
> “The page will continue to own the data fetching. The shared component will only receive normalized data and render the UI based on the provided contract. This keeps the shared component reusable and avoids coupling it to a specific vertical API.”

5.2 High-Level Architecture

```mermaid
flowchart LR
  Route[Next.js Route / Page] --> Usecase[Feature Use Case]
  Usecase --> Adapter[API Adapter]
  Adapter --> API[Backend API]

  Usecase --> ViewModel[View Model / Mapper]
  ViewModel --> Component[React Component]

  Component --> Tracking[Tracking Adapter]

```

> **Guideline**
>
> Use this diagram to show boundaries.
>
> For FE tasks, make these things clear:
>
> - Which page or route owns the feature
> - Where data comes from
> - Where mapping happens
> - Which component renders the UI
> - Where tracking is triggered
> - Whether the module is page-specific or shared

5.3 Main Design Decisions

Decision	Reason

> **Guideline**
>
> Use this table to document important decisions, not obvious implementation details.
>
> **Good decisions**
>
> - Use React Query for client-driven filter data
> - Keep shared component API-agnostic
> - Use server fetch for initial page data
> - Store template config in IndexedDB with ETag
>
> **Not worth documenting**
>
> - Use `map()` to render list
> - Use `div` for wrapper
> - Create a new file for component

---

6. Rendering & Data Fetching Strategy

Concern	Decision	Reason
Rendering mode	SSR / CSR / Static / Hybrid	
Data fetching	Server fetch / React Query / client fetch	
Cache	No cache / React Query cache / HTTP cache / IndexedDB	
Revalidation	On request / interval / manual	
Error fallback	Error boundary / fallback component / retry UI	

> **Guideline**
>
> This section is important for Next.js tasks. Do not only write the tool name. Explain why that strategy is selected.
>
> **Example**
>
> “React Query will be used because the data changes based on user-selected filters and needs client-side refetching.”
>
> **Example**
>
> “Server-side fetching will be used because the data is required for the first render and should be available before hydration.”
>
> Skip this section only if the task is purely visual and does not touch data fetching.

Optional SSR / Hydration Flow

Add this diagram if the feature mixes SSR and client-side interaction. Skip if the flow is simple.

```
sequenceDiagram
  participant User
  participant Page as Next.js Page
  participant Server as Server Fetch
  participant API
  participant Client as Client Component

  User->>Page: Open page
  Page->>Server: Fetch initial data
  Server->>API: Request data
  API-->>Server: Response
  Server-->>Page: Render HTML
  Page-->>Client: Hydrate interactive component

```

---

7. Component Design

7.1 Component Tree

```
flowchart TD
  Page[FeaturePage]
  Page --> Header[PageHeader]
  Page --> Filter[FeatureFilter]
  Page --> Content[FeatureContent]
  Content --> Card[FeatureCard]
  Page --> EmptyState[EmptyState]
```

```
  Page --> ErrorState[ErrorState]

```

> **Guideline**
>
> Show only meaningful components. Do not include every small presentational component.
>
> Focus on components that own logic, state, composition, or reusable behavior.

7.2 Component Responsibility

Component	Responsibility	State Owner	Notes

> **Guideline**
>
> Explain what each important component is responsible for. This helps prevent unclear boundaries.
>
> **Good responsibility**
>
> “Render promo list based on normalized view model.”
>
> **Bad responsibility**
>
> “Contains UI.”

---

8. Props & Component Contract

Prop	Type	Required	Default	Description

> **Guideline**
>
> Do not document every prop. Document props only when they are part of an important contract.
>
> **Elaborate props when:**
>
> - The component is reusable or part of a shared package
> - The prop affects business behavior
> - The prop is consumed by multiple pages or verticals
> - The prop has fallback/default behavior
> - The prop relates to tracking, permission, pricing, promo, eligibility, template, or API mapping
>
> **You can skip detailed prop documentation when:**
>
> - The component is internal to one page
> - The prop is obvious from TypeScript
> - The prop is only for styling, such as `className`
> - The prop does not affect business logic
>
> **Rule of thumb**
>
> TypeScript is the source of truth for the full prop type. Confluence only explains the contract that humans need to understand.

Optional Usage Example

Add usage example for shared components or complex components. Skip for small page-specific components.

```
<ComponentName
  data={data}
  placementId="SRP_CARD"
  onImpression={handleImpression}
/>

```

---

9. API Contract & Data Mapping

UI Field	Source Field	Required	Fallback	Notes

> **Guideline**
>
> This section prevents FE-BE mismatch.
>
> Do not paste the full API response if it is too long. Prefer mapping table + short sample response + link to full contract.
>
> Make these clear:
>
> - Which field is required
> - Which field is optional
> - What FE should do when data is missing
> - Which field is used for tracking
> - Which field is used for display only
> - Which field affects business behavior

Sample Response

Keep the sample response minimal. Include only fields needed to understand the FE behavior.

```
{
  "data": {}
}

```

Optional API Flow

Add this if the flow involves multiple APIs, orchestration, fallback, or transformation.

```
sequenceDiagram
  participant Page
  participant Usecase
  participant API

  Page->>Usecase: Request data
  Usecase->>API: Fetch data
  API-->>Usecase: Response
  Usecase-->>Page: Mapped view model

```

10. UI States

Scenario	Expected UI	Tracking	Notes
Initial loading			
Success with data			
Success with empty data			
API error			
Unauthorized			
Partial data missing			

> **Guideline**
>
> This section is required for almost every FE task. It helps FE, QA, and PM align on expected behavior.
>
> Always think about:
>
> - Loading
> - Success
> - Empty
> - Error
> - Permission
> - Partial or invalid data
>
> Avoid only documenting the happy path.

Optional State Diagram

Add this if the feature has multiple state transitions. Skip for simple static UI changes.

```
stateDiagram-v2
```

```
  [*] --> Loading
  Loading --> Success
  Loading --> Empty
  Loading --> Error
  Error --> Loading: Retry
  Success --> Loading: Refetch

```

---

11. Analytics & Tracking

Event	Trigger	Payload	Owner

> **Guideline**
>
> Do not only write “add tracking”. Explain event name, trigger, minimum payload, and owner or consumer of the event.
>
> **Good example**
>
> `promo_selected` is triggered when user clicks a promo option. Payload: `promoId`, `placementId`, `vertical`, `pageName`.

---

12. Performance Considerations

Concern	Decision
Bundle size	
Network request	
Cache	
Rendering	
Re-render	
Image / asset loading	

> **Guideline**
>
> Only fill what is relevant. Do not add generic best practices if they do not apply.
>
> Useful questions:
>
> - Will this increase bundle size?
> - Does this add new API calls?
> - Can the data be cached?
> - Does it need SSR?
> - Is this component imported by many verticals?
> - Can we lazy-load heavy dependencies?
>
> **Good example**
>
> “The shared module should not import page-specific dependencies to avoid increasing bundle size across all verticals.”

---

13. Security & Permission

Concern	Decision
Auth	
Permission	
Sensitive data	
Storage	
XSS	
API error exposure	

> **Guideline**
>
> For FE tasks, focus on:
>
> - Permission-based rendering
> - Token/session handling
> - Avoiding sensitive data in localStorage / IndexedDB
> - Avoiding raw HTML rendering
> - Not exposing raw backend error messages
>
> Skip details that are fully owned by BE, but mention dependency if FE behavior depends on it.

---

14. Testing Strategy

Level	Scope	Owner
Unit Test		FE
Integration Test		FE
E2E Test		FE / QA
Manual QA		QA

Test Scenarios

Scenario	Given	When	Then

> **Guideline**
>
> Use behavior-based scenario names. Test description should explain observable behavior, not implementation details.
>
> **Good**
>
> “Show fallback when selected template cannot be resolved.”
>
> **Bad**
>
> “Return false when templateId is undefined.”
>
> Think from user/system behavior:
>
> - What should user see?
> - What should happen when API fails?
> - What should happen when optional data is missing?
> - What critical flow must not break?

---

15. Rollout & Migration Plan

Phase	Scope	Rollout Strategy	Success Criteria
Phase 1			
Phase 2			
Phase 3			
Rollback			

> **Guideline**
>
> Required for risky features, shared components, API migration, or cross-vertical rollout.
>
> Mention feature flag if available. Mention rollback plan clearly.
>
> **Good rollback example**
>
> “Disable feature flag to restore old component.”
>
> **If there is no feature flag**
>
> “Rollback by reverting PR and redeploying previous stable version.”

Optional Rollout Flow

```mermaid
flowchart LR
  Dev[Dev Complete] --> QA[QA]
  QA --> Staging[Staging Validation]
  Staging --> Canary[Canary Rollout]
  Canary --> Full[Full Rollout]
  Canary --> Rollback[Rollback if issue]

```

---

16. Risks, Trade-offs, and Open Questions

Risks

Risk	Impact	Mitigation

> **Guideline**
>
> List things that can realistically go wrong. Good risks are specific.
>
> **Good**
>
> “API contract may change during development, causing FE rework.”
>
> **Bad**
>
> “There may be bugs.”

Trade-offs

Option	Pros	Cons	Decision

> **Guideline**
>
> Use this when there are multiple possible approaches. You do not need to document every small option.
>
> Focus on meaningful architectural or product trade-offs.

Open Questions

Question	Owner	Due Date	Status
			Open
			Open

> **Guideline**
>
> Use this section to make uncertainty visible. This helps reviewers focus on unresolved decisions.
>
> Every open question should have an owner.

---

17. Implementation Plan

Task	Owner	Estimate	Notes

> **Guideline**
>
> Keep this high-level. Do not duplicate Jira subtasks too much unless Jira does not exist yet.
>
> **Good**
>
> - Create API adapter and mapper
> - Build shared component
> - Integrate into SRP
> - Add unit and E2E tests
>
> **Too detailed**
>
> - Create folder
> - Create file
> - Add import
> - Rename variable

---

18. Appendix

- Full API contract:
- Full JSON sample:
- Figma:
- Storybook:
- Related ADR:
- Related incident:
- Related tech docs:
- Additional diagrams:
- Useful references:

> **Guideline**
>
> Put long details here so the main document stays clean.
>
> Examples:
>
> - Full API response
> - Full component type
> - Large diagrams
> - Related discussions
> - Previous decision records

Appendix A — Quick Rule for Filling This Template

Small FE Task

Fill only:

1. Executive Summary
2. Background & Problem
3. Goals & Non-Goals
4. Scope & Impacted Areas
5. Component Design
6. API Contract & Data Mapping, if API involved
7. UI States

8. Testing Strategy
9. Risks / Open Questions
10. Implementation Plan

Medium FE Task

Fill sections 1–17, but keep these concise:

- Performance
- Security
- Rollout
- Trade-offs

Large / Cross-Team / Shared Module Task

Fill all sections, especially:

- Proposed Solution
- Rendering & Data Fetching
- Props & Component Contract
- API Mapping
- UI States
- Performance
- Security
- Testing Strategy
- Rollout
- Trade-offs

Appendix B — AI Prompt to Generate This Document

Use this prompt when asking AI to generate a first draft.

```
Please generate a Frontend Tech Solution document using the template
below.

Context:
- Feature / initiative name:
- Business problem:
- Affected pages / journeys:
- Current behavior:
- Expected behavior:
- FE stack:
- API involved:
- Data contract / sample response:
- State management:
```

```
- Rendering strategy:
- Tracking requirement:
- Testing expectation:
- Rollout plan:
- Known risks / open questions:

Writing rules:
- Keep it straightforward and not too rigid.
- Prefer tables, diagrams, and bullet points over long paragraphs.
- Do not over-explain obvious implementation details.
- Focus on FE responsibilities, boundary, API mapping, UI states, testing, and rollout.
- Use behavior-based test scenario names.
- For React props, only elaborate props that are part of shared component contract or affect business behavior.
- If information is missing, write it as an open question instead of guessing.
- Use Mermaid syntax for diagrams where helpful.
- Keep the document readable for FE, BE, QA, PM, and future maintainers.

Template:
<paste this Confluence template here>

```

Appendix C — Diagram Decision Guide

Situation	Recommended Diagram	Why
Feature touches multiple systems	Context diagram	Shows FE, BE, shared package, tracking, and external systems
Feature has multiple page components	Component tree	Shows component ownership and composition
Feature has API orchestration	Sequence diagram	Shows API order, mapping, and fallback flow
Feature has many UI states	State diagram	Shows loading, success, empty, error, retry, and refetch transitions
Feature has phased rollout	Rollout flow	Shows canary, full rollout, and rollback

Feature has multiple teams working in parallel	Workstream diagram	Shows dependencies and parallelization

---

Appendix D — Props Documentation Rule

Document props in Confluence when the prop is part of the human contract

Examples:

Prop	Why it should be documented
`placementId`	Affects rendering, tracking, or business placement
`data`	Main source for UI rendering
`fallbackView`	Defines fallback behavior
`onImpression`	Affects analytics behavior
`isEligible`	Affects business visibility
`templateId`	Affects selected UI template

Do not over-document obvious implementation props

Examples:

Prop	Why it can usually be skipped
`className`	Styling-only and obvious
`children`	Common React pattern
`id`	Obvious unless it affects tracking or business logic
`isOpen`	Obvious unless it has special behavior

Recommended Confluence level

Use Confluence for important contract explanation. Use TypeScript or Storybook for the full API reference.

Appendix E — Quality Checklist Before Review

Use this checklist before asking others to review.

Check	Done
Problem is clear before solution	
Goals and non-goals are explicit	
Impacted pages/modules/services are listed	
Proposed solution explains boundary and ownership	
Rendering and data-fetching strategy is clear	
Component responsibility is not vague	
Important props are documented, but not over-documented	
API mapping explains required, optional, and fallback fields	
Loading, empty, error, and partial-data states are covered	
Tracking events have trigger and payload	
Performance concerns are listed only when relevant	
Security and permission behavior are covered when relevant	

Test scenarios use behavior-based wording	
Rollout and rollback plan are clear for risky changes	
Risks, trade-offs, and open questions are visible	